

Final Review Sheet

Generated from the Markdown study system. Labs with executable code remain in the `labs/` folder. This is study material, not a Cisco exam dump and not a pass guarantee.

CCNA Automation 200-901 v1.1 — Final Review

Use this in the last 3–5 days. It is a memory sheet, not a textbook. Teach-back from `CCNAAUTO_COMPLETE_STUDY_GUIDE.md` if any line here is fuzzy.

This is not a dump of real exam items.

HTTP methods (REST)

Method	Typical meaning	Idempotent?
GET	Read	Yes
POST	Create / invoke	No
PUT	Replace resource	Yes
PATCH	Partial update	Should be, not always
DELETE	Remove	Yes
HEAD	Headers only	Yes
OPTIONS	Allowed methods	Yes

RESTCONF uses the same verbs on YANG data. NETCONF uses XML RPCs instead (`get` , `get-config` , `edit-config` , `commit`).

HTTP response codes

Code	Meaning	First thing to check
200 OK	Success with body	Parse JSON/XML
201 Created	Resource created	Location header
204 No Content	Success, empty body	DELETE/PUT often
400 Bad Request	Malformed body/query	Schema vs docs
401 Unauthorized	Not authenticated	Password, token, API key
403 Forbidden	Authenticated, no permission	Role/scope
404 Not Found	Bad path or missing resource	URL / ID
409 Conflict	State conflict	Duplicate, etag, lock
415 Unsupported Media Type	Wrong Content-Type	<code>application/json</code> or <code>yang-data+json</code>
429 Too Many Requests	Rate limit	Retry-After
500 Internal Server Error	Server fault	Retry; don't assume payload
503 Service Unavailable	Down / maintenance	Retry later

401 = who are you? 403 = we know who you are, you may not do that.

API authentication

Mechanism	How it appears
Basic	<code>Authorization: Basic base64(user:pass)</code>
API key	Custom header, e.g. <code>X-Cisco-Meraki-API-Key</code>
Bearer / custom token	<code>Authorization: Bearer <token></code> or <code>X-Auth-Token</code> (Catalyst Center)

Catalyst Center: `POST /dna/system/api/v1/auth/token` with Basic, then send `X-Auth-Token`.

Meraki: API key on every request.

Webex: Bearer token.

REST vs RPC vs sync/async

- **REST:** resources, URLs, HTTP verbs, JSON, stateless.
 - **RPC:** call a function (`/getDeviceList` , JSON-RPC, NETCONF `<get-config>`).
 - **Synchronous:** client waits for the result.
 - **Asynchronous:** 202 + job URL, webhook, or callback. Webhooks are async notifications.
-

Webhooks

Provider POSTs to *your* HTTPS URL when an event happens. You subscribe, verify (secret/signature or challenge), return 2xx quickly, process later. Retries are normal. Opposite of polling.

API constraints

Rate limits, pagination, max payload, API version in path/header, TLS certificates, required scopes, idempotency, timeouts. 429 is the exam-friendly rate-limit signal.

Data formats

	JSON	XML	YAML
REST	Default	Occasional	Rare as body
NETCONF	No (XML)	Yes	No
RESTCONF	yang-data+json	yang-data+xml	No
Ansible	Possible	No	Native
Comments	No	<code><!-- --></code>	<code>#</code>

Python: `json.loads` , `yaml.safe_load` , `xml.etree.ElementTree` .

Python requests skeleton

```
import requests
r = requests.get(
    "https://api.example.com/devices",
    headers={"Accept": "application/json", "X-API-Key": key},
    params={"limit": 10},
    timeout=30,
)
r.raise_for_status()
data = r.json()
```

json= sets body + Content-Type. auth=(user, pass) is Basic. verify=False is lab-only for self-signed certs.

Git

```
working tree → git add → staging → git commit → local repo → git push → remote
git pull = fetch + merge (or rebase)
```

Command	Purpose
git clone URL	Copy remote repo
git status	What changed
git add FILE / git add -A	Stage
git rm FILE	Remove and stage
git commit -m "msg"	Snapshot
git branch / git switch -c NAME	List / create branch
git merge BRANCH	Integrate
git diff	Unstaged vs HEAD
git diff --staged	Staged vs HEAD
git push / git pull	Remote sync
git log --oneline	History

Conflict: <<<<<<< == >>>>>>> — edit, git add, git commit.

Unified diff

```
--- a/old
+++ b/new
@@ -1,3 +1,4 @@
  unchanged
-removed
+added
```

```
@@ -oldStart,oldCount +newStart,newCount @@
```

YANG / NETCONF / RESTCONF

Piece	Role
YANG	Model (what data exists)
NETCONF	Protocol: XML RPC over SSH :830
RESTCONF	Protocol: HTTP on YANG over TLS :443

NETCONF RPCs: get, get-config, edit-config, copy-config, delete-config, lock, unlock, commit, discard-changes, close-session.

Datastores: running (active), candidate (stage + commit), startup (boot). Operational state via `get / config false`.

RESTCONF URL: `/restconf/data/{module}:{container}/{list}={key}`

Headers: `Accept: application/yang-data+json`

YANG nodes: module, namespace, prefix, container, list + key, leaf, leaf-list, `config false` = operational.

Docker

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
ENV PORT=8080
EXPOSE 8080
CMD ["python", "app.py"]
```

`FROM` base image. `RUN` executes at build. `CMD / ENTRYPOINT` at runtime. Each instruction is a layer.

```

docker build -t name:tag .
docker images
docker run --rm -p 8080:8080 name:tag
docker ps
docker logs
docker exec -it CONTAINER cmd
docker stop / rm / rmi

```

Bash

```

pwd; ls -la; cd /tmp
mkdir -p labs && cp a.txt b.txt && mv b.txt c.txt
rm -rf olddir
export TOKEN=secret
echo $PATH
chmod 600 .env
cat file | grep error
sudo apt install nginx      # Debian/Ubuntu

```

Ansible vs Terraform vs NSO

	Ansible	Terraform	NSO
Style	Task list (often idempotent modules)	Declarative HCL + state	Service models + device models
Agent	Agentless (SSH/API)	Providers/APIs	Orchestrator with southbound drivers
Exam verb	Interpret playbooks	Describe capabilities	Describe capabilities
Typical	package, user, copy, service	resource, plan, apply	commit services to many devices

Playbook skeleton: `hosts` , `become` , `vars` , `tasks` with `package` / `user` / `copy` / `service` .

Terraform: `terraform init/plan/apply/destroy` . State file records real resources.

Controller vs device

- **Device-level:** SSH, NETCONF, RESTCONF, NX-API to one box.
- **Controller-level:** Meraki dashboard, Catalyst Center, APIC, vManage, NSO — inventory, intent, fabric, overlay.

Cisco platform one-liners

Platform	API purpose
Meraki	Cloud-managed networks; key header; orgs/networks/devices/clients
Catalyst Center	Campus intent; token then /dna/intent/api/v1/network-device
ACI / APIC	Data-center fabric object model; aaaLogin
Catalyst SD-WAN	Overlay WAN via vManage
NSO	Multi-vendor service orchestration
UCS Manager	Blade/rack UCS XML API
Intersight	Cloud ops for compute
Webex	Rooms, messages, memberships, people; Bearer
Webex devices	xAPI / cloud device control
CUCM AXL	Admin SOAP/XML
CUCM UDS	User-facing REST
XDR	Correlate incidents across security products
Firepower	FMC policy/event REST
Secure Connect	SASE / cloud security access APIs
Secure Endpoint	Endpoint detections
ISE	Identity policy; ERS / pxGrid
Secure Malware Analytics	Detonate files/URLs
IOS XE	NETCONF/RESTCONF/gNMI, Guest Shell
NX-OS	NX-API CLI (JSON-RPC), NX-API REST, often NETCONF/RESTCONF

DevNet: which resource?

Need	Use
Live gear/API	Sandbox
Sample code	Code Exchange / GitHub
Guided tutorial	Learning Labs
Request schema	API documentation
Peer help	Community forums
TAC-style product bugs	Support (with contract)

CI/CD, IaC, DevOps

- **CI:** merge → automatically build and test.
- **CD:** automatically deliver / deploy an artifact.
- **IaC:** desired state in Git; repeatable; reviewable; no snowflake boxes.
- **DevOps:** culture + automation + measurement + sharing (CALMS). Not “install Jenkins.”

App pipeline: commit → build → unit test → image → registry → deploy → monitor.

Infra pipeline: same idea, but the artifact is config/plan (Ansible/Terraform/NSO).

Security

- **Secrets:** env vars, vault, never Git. Rotate keys.
 - **In transit:** TLS (HTTPS, SSH, NETCONF over SSH).
 - **At rest:** disk/volume encryption, encrypted backups.
 - **XSS:** inject script into pages; escape output.
 - **SQLi:** untrusted string in query; use parameters.
 - **CSRF:** trick a logged-in browser into a state-changing request; tokens / SameSite.
-

App front-door

DNS → firewall → load balancer → reverse proxy → app

DNS names to IPs. Firewall allows ports. LB spreads connections and health-checks. Reverse proxy terminates TLS, routes `/api` vs `/`.

Deployment models / types

Model	Who owns infrastructure
Private cloud	You (or dedicated hosted private)
Public cloud	Provider, shared hardware, your tenant
Hybrid	Both, connected
Edge	Compute near users/devices; low latency

Type	Isolation	Density
Bare metal	Whole server	Low
VM	Hardware virtualization	Medium
Container	Shared kernel	High

Network planes

Plane	Job	Examples
Data	Forward packets	ASIC, CEF
Control	Build forwarding info	OSPF, BGP, ARP
Management	Operate the box	SSH, HTTPS, NETCONF, SNMP, syslog

Ports (memorize)

Protocol	Port
SSH	22/tcp
Telnet	23/tcp
HTTP	80/tcp
HTTPS / RESTCONF (typical)	443/tcp
NETCONF	830/tcp
DNS	53/udp (and tcp)
DHCP	67/udp server, 68/udp client
NTP	123/udp
SNMP	161/udp (query), 162/udp (trap)

IP services (one line each)

- **DHCP:** lease IP, mask, gateway, DNS.
 - **DNS:** name → IP (and reverse).
 - **NAT:** rewrite addresses; PAT overload; breaks inbound without mapping.
 - **SNMP:** monitor/manage devices (UDP).
 - **NTP:** time sync; needed for certs, logs, correlation.
-

MAC, VLAN, IP

- **MAC:** 48-bit L2 identity; switches learn/flood/forward.
- **VLAN:** L2 segment; same VLAN = broadcast domain; inter-VLAN needs L3.
- **IP:** L3 identity. Prefix `/24` = mask `255.255.255.0`.
- **Gateway:** local router IP for off-subnet traffic.
- **Route:** prefix + next-hop or exit interface.

Usable hosts: `/24` → 254, `/30` → 2, `/32` → 1.

Devices

- **Switch:** L2 (and often L3 SVIs).
 - **Router:** L3 forwarding between networks.
 - **Firewall:** policy allow/deny, often NAT/VPN.
 - **Load balancer:** distribute to a pool; health checks.
-

Connectivity diagnosis (6.8)

Symptom	Likely cause
App sees LB/firewall IP as client	NAT / SNAT
SYN, no SYN-ACK	Port blocked
Works off-net, fails on corp Wi-Fi	Proxy / SSL inspection
Tunnel up, overlapping 10.x	VPN interesting-traffic / overlap

Network constraints on apps (6.9)

Latency, jitter, loss, bandwidth, MTU/fragmentation, DNS failure, TLS intercept, chatty APIs over high RTT. Automation that times out is often a network constraint, not a Python bug.

TDD / unittest

Red (write failing test) → green (minimal code) → refactor.

```
python -m unittest test_subnet.py . assertEquals , assertRaises .
```

MVC / Observer

- **MVC:** Model (data), View (presentation), Controller (input/API).
 - **Observer:** subject notifies subscribers. Webhooks are Observer over HTTP.
-

Agile / Lean / Waterfall

- Waterfall: sequential phases, late feedback.
 - Agile: short iterations, working software, changing requirements.
 - Lean: remove waste, small batch, fast flow.
-

Frequently confused

Pair	Difference
401 vs 403	Auth missing vs permission missing
PUT vs PATCH	Replace vs partial
GET vs POST	Read vs create/action
JSON vs YAML	YAML allows comments and indentation; JSON does not
NETCONF vs RESTCONF	XML/SSH/830 vs HTTP/443; same YANG
get vs get-config	State+config vs config only
running vs candidate	Live vs staged
Ansible vs Terraform	Tasks vs declarative state
NSO vs Ansible	Service orchestrator vs general automation engine
AXL vs UDS	CUCM admin SOAP vs user REST
UCS Manager vs Intersight	On-prem UCS domain vs cloud ops
Catalyst Center vs Meraki	On-prem intent controller vs cloud dashboard
Reverse proxy vs LB	Routing/TLS vs spreading load (often combined)
Encoding vs model	JSON/XML vs YANG
Clone vs pull	Initial copy vs update
Merge vs rebase	Combine histories vs replay commits
VM vs container	Guest OS vs shared kernel

Last-day drill

1. Recite ports 22, 23, 80, 443, 830.
2. Draw YANG → NETCONF/RESTCONF → datastores.
3. Read one Ansible playbook and one Terraform file aloud.
4. Given 401/403/404/415/429, state the fix.
5. Write a 10-line `requests` GET with headers and `.json()`.
6. Read a unified diff.
7. Name the Cisco platform that lists wireless clients in the cloud (Meraki) vs campus controller (Catalyst Center).